17. Policy Gradients Actor-Critic

Overview

Policy gradient methods optimize expected return directly. REINFORCE, advantage estimation, actor to critic architectures, and A2C synchronous updates are derived and connected to variance reduction techniques used in modern RL.

Lab: Lab 17.

Prior modules: Module 15, Module 16.

Contents

Module track

This module builds on **Module 15**, **Module 16**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

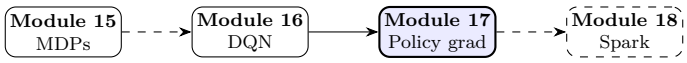

Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Policy-Based Methods

Value-based DQN (**Module 16** (DQN loss)) struggles with continuous or stochastic action spaces. Policy gradients optimize parameterized policy $\pi_\theta(a|s)$ directly [4].

2 Policy Gradient Theorem

Objective $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$ gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\right], \tag{1}$$

where $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ is return.

Note. REINFORCE uses full Monte Carlo returns, high variance, unbiased.

3 Variance Reduction

Subtract baseline $b(s_t)$ without bias:

$$\nabla_\theta J \approx \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t|s_t)\,(G_t - b(s_t))\right]. \tag{2}$$

Learned value baseline leads to actor to critic.

4 Actor to Critic

Actor updates π_θ; critic estimates $V_\phi(s)$ or $Q_\phi(s,a)$:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \qquad \text{(TD error)}, \tag{3}$$

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t|s_t)\, \delta_t, \tag{4}$$

$$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}_{\text{critic}}. \tag{5}$$

5 Advantage Actor to Critic (A2C/A3C)

Advantage $A_t = G_t - V(s_t)$ reduces variance vs. raw return in (1). PPO clips policy ratio $r_t(\theta) = \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t(\theta)A_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t\right)\right]. \tag{6}$$

6 Entropy Bonus

Encourage exploration: add $-\beta \sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)$ to loss.

Table 1: Policy vs. value methods.

Aspect	DQN	PPO
Action space	Discrete	Discrete + continuous
Sample efficiency	Moderate (replay)	On-policy
Stability	Replay + target net	Clipped objective

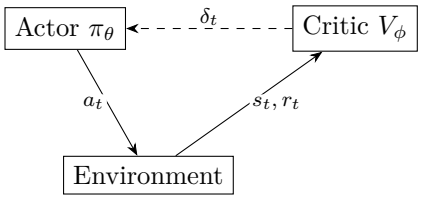

Figure 2: Actor to critic feedback loop.

7 Lab

Lab 17 trains PPO on a continuous control task; compare sample efficiency to DQN in **Module 16** (experience replay).

8 Trust Region Methods

TRPO constrains KL divergence between old and new policy:

$$\max_\theta \mathbb{E}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_t\right] \text{ s.t. } \mathbb{E}[\text{KL}(\pi_{\theta_{\text{old}}} \| \pi_\theta)] \leq \delta. \tag{7}$$

PPO approximates TRPO with clipped surrogate from section 5.

8.1 Continuous Actions

Gaussian policy $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma^2_\theta(s))$ for robotics control.

9 Extended Intuition

Policy gradient methods optimize expected return $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G_0]$ directly by ascending $\nabla_\theta J$. REINFORCE uses Monte Carlo returns and updates θ with $\nabla_\theta \log \pi_\theta(a|s) \cdot G_t$, which has high variance. Baselines subtract a value estimate $b(s)$ from G_t without biasing the gradient in expectation: use advantage $A_t = G_t - V(s_t)$.

Actor-critic maintains two networks: actor $\pi_\theta(a|s)$ and critic $V_\phi(s)$ (or $Q_\phi(s,a)$). A2C runs multiple workers synchronously, averaging gradients before each update (contrast with async A3C). PPO clips probability ratios to prevent destructively large policy updates; common in modern continuous control.

DQN (**Module 16**) excels in discrete action spaces; policy gradients handle continuous torque/velocity outputs natively. Variance reduction (advantage normalization, entropy bonus) often matters more than architecture width on MuJoCo-scale tasks.

10 Numerical Walkthrough

CartPole with policy net softmax over 2 actions, value net scalar output, $\gamma = 0.99$, lr $= 3 \times 10^{-4}$, 5-step rollout.

Episode return $G_0 = 45$; at step t, $\log \pi_\theta(a_t|s_t) = -0.693$ (prob 0.5 each action). REINFORCE update direction $\approx -0.693 \cdot 45 \cdot \nabla_\theta \log \pi$ (sign via loss minimization formulation).

With baseline $V(s_t) = 20$, advantage $A_t = G_t - 20$ reduces magnitude for critic-fitted states. Entropy bonus $-\beta \sum \log \pi(a|s)$ with $\beta = 0.01$ encourages exploration; entropy 0.693 for uniform 2-action.

A2C 4 parallel envs, batch 20 steps each: 80 transitions per update; total 500k steps to solve CartPole reliably. PPO clip $\epsilon = 0.2$ limits ratio $r_t(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$ to $[0.8, 1.2]$ in objective.

11 PyTorch Implementation Notes

- Policy: `nn.Linear` head + `Categorical(logits)` from `torch.distributions`.
- Sample action: `dist.sample()` ; log prob: `dist.log_prob(action)` for REINFORCE loss `-(log_prob * advantage).mean()`.
- Critic loss: `F.mse_loss(V(s), returns)` or TD residual for one-step bootstrap.
- `stable-baselines3` PPO: `PPO("MlpPolicy", env, n_steps=2048, batch_size=64, learning_rate=3e-4)`.
- Normalize advantage across batch: `(adv - adv.mean()) / (adv.std() + 1e-8)`.

```
logits = policy(s)
dist = Categorical(logits=logits)
a = dist.sample()
loss_pg = -(dist.log_prob(a) * advantage.detach()).mean()
loss_v = F.mse_loss(critic(s), returns)
```

12 Local GPU Constraints

CartPole A2C/PPO trains fine on CPU; SB3 uses GPU optionally for Atari CNN policies. Continuous control (Pendulum) MLP fits 4 GB easily with vectorized envs.

Parallel envs increase CPU load more than GPU; on a laptop, 4 to 8 env workers is typical. Long horizon tasks need more rollout steps per update; adjust `n_steps` before widening networks.

13 Debugging Checklist

- Policy entropy collapses to 0: remove entropy coef or increase β; lr too high on actor.
- Critic loss diverges: returns not normalized on long episodes; try reward scaling or γ check.
- REINFORCE flat performance: need baseline/critic; raw returns too noisy.
- PPO KL spikes: clip range too loose or too many epochs on same batch (`n_epochs`).
- NaN logits: observation not clipped in Atari; enable SB3 `VecNormalize`.

13.1 PPO Objective, GAE, and Continuous Actions

PPO replaces raw policy gradient steps with a clipped surrogate so large policy updates do not collapse performance in one batch. The probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is clipped in the objective:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]. \tag{8}$$

With $\epsilon = 0.2$, ratios outside $[0.8, 1.2]$ contribute zero gradient, preventing destructive updates when advantage is positive. Generalized Advantage Estimation (GAE) blends multi-step returns with TD residuals:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \tag{9}$$

$\lambda = 0.95$ is a common default; higher λ lowers bias, raises variance.

Continuous control (Pendulum, MuJoCo) uses Gaussian policy $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$; REINFORCE backprops through sampled actions via reparameterization or score-function estimator. Action bounds are enforced by squashing through tanh and scaling to env limits; log-prob correction terms must be included or entropy collapses at boundaries. Entropy coefficient β in total loss $\mathcal{L} = \mathcal{L}^{\text{CLIP}} - \beta\mathcal{H}(\pi) + c_v\mathcal{L}_V$ trades exploration vs exploitation; decay β slowly on easy tasks, keep fixed on hard sparse-reward tasks. Multiple epochs (`n_epochs=4 to 10`) on the same rollout batch improve sample efficiency but increase overfitting risk; watch KL divergence between old and new policy (target KL 0.01 to 0.05). Value function clipping in PPO mirrors policy clipping and stops critic spikes from dominating the combined loss when returns are heavy-tailed. Policy gradients handle continuous spaces where DQN (**Module 16**) would need discretization; discretizing torque into 50 bins often fails on fine control tasks. Trust Region Policy Optimization (TRPO) constrains KL divergence explicitly; PPO achieves similar stability with simpler clipped objective and is the default in most SB3 configs. Importance sampling corrections matter for off-policy data in actor-critic variants; PPO stays on-policy by reusing rollouts only for a few epochs before collecting fresh trajectories. Reward normalization (`VecNormalize` in SB3) standardizes returns and observations; critical for MuJoCo where raw rewards span orders of magnitude across environments. Log `explained_variance` of the value function; values below 0.5 indicate the critic is not tracking returns and policy updates become noisy. Deterministic evaluation: set `deterministic=True` in SB3 `predict` after training; stochastic sampling inflates variance on reporting final return. Action distribution entropy should stay above 0.3 nats early in CartPole; below 0.1 before solve threshold means premature exploitation. Continuous action std parameterization: learn $\log\sigma$ for stability; clamp σ to $[0.01, 2]$ to avoid collapse or explosion. On-policy methods cannot reuse old trajectories after many policy updates; if reusing data, switch to off-policy SAC/TD3 (outside lab scope) or collect fresh rollouts each epoch. Seed SB3 with `seed=42` and log `monitor.csv` return column; compare PPO runs only when env seed and `n_steps` match exactly.

14 Practice Problems

Problem 1. Show subtracting baseline $b(s)$ from return does not change expected policy gradient (sketch).

Problem 2. Train REINFORCE vs A2C on CartPole; compare steps to solve and variance of returns.

Problem 3. Tune PPO `clip_range` 0.1 vs 0.3 on Pendulum; plot learning curve.

Problem 4. Log actor entropy during training; correlate collapse with performance drop.

Run **Lab 17**; compare sample efficiency to DQN in **Module 16**; MDP basics in **Module 15**.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 17.

Topic	Primary source	Where in this PDF
Policy gradient theorem	[5]	Eq. 1
Actor-critic / A3C context	[1]	Section 5
PPO reference	[3]	Extension section
Value-based contrast	[4]	Overview

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

[1] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, et al. Asynchronous methods for deep reinforcement learning. In *ICML*, 2016.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, et al. Proximal policy optimization algorithms. 2017.

[4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[5] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. volume 8, pages 229–256, 1992.